

Understanding Your Own Project

ClaimDesk

You know manual testing, test cases and bug reports. You may have written some Selenium. This guide assumes that and nothing more. Every term that is not plain English is explained the first time it appears, and every code sample is taken from your repository, not invented for the guide.

Read sections 1 to 3 to be able to describe the project. Read 4 and 5 to be able to defend it in an interview.

1. The 60-second version

If someone asks what you built, this is the answer. It is worth learning close to word for word, because the follow-up question is always the interesting one.

Say this

ClaimDesk QA is a test automation framework in Python. It tests an insurance-claims web application through three independent layers: the browser with Playwright, the REST API with httpx, and the PostgreSQL database with a read-only role. 351 tests. It runs on GitHub Actions as a pull-request gate and a nightly cross-browser matrix, inside Docker Compose, and on Jenkins. The application is only a fixture. The framework is the deliverable.

The last sentence is the one that matters. Most portfolio projects are an app with a few tests bolted on. Yours is the opposite: the app exists so the test framework has something realistic to bite. That is what an SDET job actually is.

2. The two halves of the repository

app/ - ClaimDesk, the application under test

A working FastAPI web application with a PostgreSQL database: customers file claims, adjusters review them, admins manage users. It has real login, three roles, a status workflow, money limits, an audit trail and a payout ledger. It even has its own 58 tests.

You did not build it to ship it. You built it so the framework has something honest to test. Think of it as a crash-test dummy: realistic enough that the crash means something.

src/claimdesk_qa/ - the framework, and the actual deliverable

This is an installable Python package, 43 files and about 3,900 lines. It is not a folder of test scripts. It is a library, with its own unit tests, that knows how to drive and inspect ClaimDesk.

tests/ - the tests themselves

32 files, about 5,500 lines, split by how they attack the application:

tests/framework/	130 tests	the framework's own unit tests (no app needed)
tests/api/	157 tests	REST API
tests/ui/	32 tests	real browser
tests/db/	28 tests	database state
tests/e2e/	4 tests	one journey across all three

Why 130 tests of the framework itself?

This is the part most people miss and the part interviewers notice. Your framework contains real logic: retry policy, artifact naming, log masking, readiness polling. That logic is software, and untested software in a test framework means a green suite might be green for the wrong reason. Testing your own tools is the difference between a script collection and an engineering product.

3. What actually happens when a test runs

Follow one API test from the command to the result. This is the whole machine in one page.

1. You run pytest. It reads pyproject.toml, which turns on strict settings:

```
--strict-markers    # a typo'd marker fails the run instead of silently
                   # matching nothing
--strict-config     # a typo'd config key fails the run
```

That matters more than it looks. Without strict markers, writing `@pytest.mark.smok` instead of `smoke` means the test quietly drops out of the smoke suite and nobody finds out.

2. Fixtures run before the test. A fixture is just setup code that pytest injects for you:

```
def test_a_customer_can_sign_in(anonymous_page, settings, password):
    ^^^^^^^^^^^^^^^^^^^^^^^^^
    # pytest sees the name, finds the fixture, runs it, hands you the result
```

3. The test asks the application to do something, through a service object rather than raw HTTP. 4. Every request carries an X-Request-Id header so the test can later be matched to the application's own log lines. 5. Everything that happened is kept in memory. 6. If the test passes, that memory is thrown away. If it fails, it is written to artifacts/ and attached to the report.

Evidence on failure only

Recording everything always would slow the suite and bury the one failure in noise from 350 passes. Recording nothing means a 3am CI failure you cannot reproduce. Keeping it in memory and only writing it on failure gives you both.

4. The building blocks, with real code from your repo

These four patterns are 90 percent of the framework. If you understand these, you understand the project.

4.1 Page Object - for the browser

A class that represents one screen. Tests say what they want to do; the class knows where the buttons are. When the UI changes, you edit one file instead of forty tests.

src/claimdesk_qa/ui/pages/login_page.py

```
class LoginPage(BasePage):
    path = "/login"

    @property
    def email_field(self) -> Locator:
        return self._page.get_by_test_id("email")

    def sign_in(self, email: str, password: str) -> Self:
        self.email_field.fill(email)
        self.password_field.fill(password)
        self.submit_button.click()
        return self
```

The detail worth remembering

`sign_in()` deliberately does NOT assert that login succeeded. About half the tests using it expect failure. A method that asserted success would force every negative test to reach around it, and a page object people work around stops being maintained. Say this in an interview and you will sound like someone who has maintained a suite, not just written one.

Also note `get_by_test_id`. Locating by a dedicated test id rather than by CSS class or text means a visual redesign or a wording change does not break the test.

4.2 Service Object - for the API

The same idea, one layer down. Tests call a method; the class knows the URL and the parameters.

src/claimdesk_qa/api/services/claims.py

```
class ClaimsApi:
    """Wraps /claims."""

    def list(self, *, status=None, min_amount=None, page=None, ...):
        return self._client.get("/claims", params={...})
```

Same design rule as the page object: these methods return the raw response instead of a parsed model, so a test expecting 200 and a test expecting 403 call exactly the same method.

4.3 Query Object - for the database

The database equivalent. A test says what it wants to know, not how to find out. Compare these two:

```
events = claim_events.for_claim(claim.id)          # what you write

cursor.execute(                                   # what you avoid
    "SELECT ... FROM claim_events WHERE claim_id = %s ORDER BY ..."
)
```

SQL lives in one place, so a column rename is one edit. Tests stay readable. And every statement is parameterised by construction, because no path through that module builds SQL by joining strings together.

Every query is a SELECT, and could not be anything else

The database login the tests use holds SELECT permission and nothing else, and the connection is opened read-only. So a test cannot insert a row to set up a scenario, even by accident. That forces test data to be created the way a real user would create it: through the application. Data the application could never produce cannot sneak into your tests.

4.4 Fixture - shared setup, injected by name

Fixtures are pytest's dependency injection. You declare what you need as an argument name and pytest builds it.

tests/_fixtures/identities.py

```
@pytest.fixture(scope="session")
def role_tokens(settings, app_ready) -> dict[str, str]:
    """One login per role for the whole session."""
```

scope="session" means it runs once for the entire test run, not once per test, because a bcrypt password check costs real time and a token does not change. But the HTTP clients built from those tokens are created fresh per test, so no state leaks between tests. Knowing why each fixture has the scope it has is exactly the kind of thing you get asked.

5. The one big idea: the black-box boundary

If you remember one thing from this document, make it this. It is the strongest idea in the project and the most likely interview question.

The rule: the test code is never allowed to import the application's code.

Your framework keeps its own copy of the application's business vocabulary - the claim statuses, the roles, the legal transitions - in `src/claimdesk_qa/domain.py`, duplicated on purpose.

src/claimdesk_qa/domain.py - the framework's own copy

```
class ClaimStatus(StrEnum):
    DRAFT = "DRAFT"
    SUBMITTED = "SUBMITTED"
    APPROVED = "APPROVED"
    ...
```

Why duplicate instead of import?

Importing would be shorter, and it would make every contract regression invisible. Rename the value "APPROVED" to "Approved" in the application, and if both sides share one definition they move together: your tests stay green while every real API consumer breaks. The copy in the framework is the specification - what you assert the API promises. The application's copy is the implementation. When they disagree, a test fails. That is the entire point.

The rule is enforced three ways, none of which rely on anyone remembering it:

1. A linter rule that refuses the import and fails the build.
2. A database role with read permission only, so tests physically cannot fake data.
3. A Docker test image that does not contain the application at all. CI proves it by checking that import `claimdesk` fails inside the image.

what the linter prints if you try

```
TID251 `claimdesk` is banned: The test framework must NEVER import
the application under test.
--> tests/api/_ban_probe.py:1:1
```

6. The clever bits, and the problem each one solves

Each of these is a small module in `src/claimdesk_qa/core/`. Each exists because of a specific failure. This section is your answer to "what was hard?"

Session injection instead of logging in

Almost every browser test needs to be signed in, and almost none are about signing in. Driving the login form in each one means every test pays for a page load, two fills, a form post, a redirect and a bcrypt check - and every one of them turns red when login breaks. One defect, fifty red tests, real cause buried. So the session is injected directly, and only the five login tests actually drive the form.

Readiness polling instead of sleep

The classic way a containerised suite becomes flaky is `time.sleep(10)` with a comment saying 'should be enough'. It is too long on a fast machine and too short on a loaded CI agent. The framework polls a `/health/ready` endpoint until the app answers, then proceeds immediately.

Correlation IDs

Every request carries an X-Request-Id derived from the test's name. The application echoes it and logs it. When something fails in CI you grep the application log for one id and see exactly the requests that test made, instead of guessing from timestamps.

One shared TLS context

httpx builds a fresh SSL context per client, and building one means parsing a few hundred KB of certificates. The framework creates a client per identity per test, so that cost was being paid on nearly every test. Sharing one context took the suite from 107 seconds to 30.

A clock that does not trust the machine

The app rejects future incident dates, and the suite checks both sides of that boundary. Both used the local date of whichever machine asked. Fine on one machine. Then Docker made them two: a UTC container and an IST runner. Between midnight and 05:30 the runner is a day ahead, so the test would fail for five and a half hours a day, in one environment only. Both sides now answer in UTC explicitly.

7. The bugs it actually found

This is your best interview material, because these are real and you can explain the reasoning behind each one.

Q. A test that passed for the wrong reason

A. GET /cl
httpx keep
request. Th
own cookie

Q. A 3.5x slowdown that was not the app's fault

A. Every A
product, be

Q. Reporting where every failure category matched nothing

A. Allure n
Triage had
the regex a

Q. The same concurrency bug, written twice

A. Two tes
came out o
an aggrega

8. The numbers, and what they mean

Every one of these came from a command that was run, not an estimate.

351 tests	130 framework, 157 API, 32 UI, 28 DB, 4 end-to-end
24.88 s	running one at a time (median of 3)
16.82 s	with 4 parallel workers = 1.48x faster
19.03 s	with 8 workers - SLOWER than 4
23.81 s	with 16 workers - barely better than serial
0 flakes	across 10 consecutive runs (3,510 executions)
94 files	checked by mypy in strict mode

The finding hiding in that table

The parallelism curve turns around. -n auto is the setting everyone reaches for, and here it throws away nearly the whole benefit: 23.81s against 16.82s at four workers. The suite is I/O-bound and the application is a single process, so past a point you are adding contention, not speed. 'Use all your cores' is a guess, and it is measurable in ten minutes.

Be careful with one of these in interviews: the same suite measured 32.77s in an earlier phase and 24.88s later, on the same machine, with nothing to explain it. A 24 percent swing from nothing at all, bigger than the gap between two and four workers. That is why every figure is published as a median with its spread and its hardware. The TLS number (107s to 30s) is the one claim made without hedging, because the cause was measured directly rather than inferred from two suite runs.

9. How to run it

Quickest route, needs only Docker:

```
docker compose --profile test -f docker/docker-compose.yml build
docker compose -f docker/docker-compose.yml up -d db sut
docker compose -f docker/docker-compose.yml run --build --rm tests
```

On your own machine, in PowerShell:

```
py -3.12 -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -e ".[dev,app]"
Copy-Item .env.example .env          # then set the two passwords
.\scripts\local_db.ps1 start         # disposable database
python -m uvicorn claimdesk.main:app --app-dir app --port 8000
```

Then in a second terminal:

```
pytest -m framework -q      # 130 tests, needs nothing else running
pytest -m api -q            # 157 API tests
pytest -q -n 4              # everything, 4 workers - the fast way
```

If pip fails with a certificate error, PostgreSQL has set CURL_CA_BUNDLE to a path that does not exist. Run: Remove-Item Env:CURL_CA_BUNDLE

10. Questions you will be asked

Q. Why not just use Selenium?

A. Playwright is better than Selenium by step; and it's easier to use.

Q. Why test the same thing at three layers? Is that not duplication?

A. They are different layers. It's not that a person is testing the same thing saved. A browser is testing the same thing.

Q. Why is the database role read-only?

A. So a test can't change the information. It's a read-only role. It's sixteen for the pg_catalog.

Q. How do you keep parallel tests from interfering?

A. Test data is isolated. It cannot be shared. It's invariants that are tested.

Q. What would you do next?

A. Two things. One is because of the fallback rate. It's measured.

11. If you only remember five things

1. The app is a fixture. The framework is the deliverable.
2. The framework never imports the app. It keeps its own copy of the rules, so a contract break fails a test instead of hiding.
3. Page objects, service objects and query objects are the same idea at three layers: say what you want, not how to get it.
4. Evidence is captured on failure only, and correlation ids tie a failing test to the app's own logs.
5. Every number is measured and published as a median with its hardware. Where something is unproven, the log says so.

That last one is the thread running through the whole project, and it is what makes the rest believable.